MS2 Summary Predicting One Piece Card Game Leader Matchups

Kiana Kiser | HS26 | I.BA_MLOPS | GitHub Repository: github.com/kianakiser/optcg-swiss-forecast

1 Summary of the milestone

What I built: My pipeline turns finished One Piece Card Game matches into a table. One row holds both Leader card numbers and one result: 1 if the first listed player won and 0 if the other player won. Both Leaders' games count towards their records.

Which games I used: I started with eight tournaments and checked them individually. They span June to September 2025 and contain 1,290 decided matches. I used these games to test the pipeline. To say which Leaders are strongest today, I'd need more recent results. The daily job checks finished tournaments on Limitless, the platform where I get the results.

What I calculate: Before each tournament date, I count each Leader's earlier games and wins, both overall and in the past 28 days. I also count how often these exact Leader cards faced each other. If two tournaments take place on Sunday, I use results dated Saturday or earlier for both. The archive tells me when a tournament took place, but sometimes I can't tell when its results went online.

What worked: I ran the daily job locally. It collected live results, built the table and saved it in Hopsworks. I compared the saved copy with the table that was calculated on my Mac. All 1,290 rows matched. I also ran the job from an empty folder. It could still retrieve earlier matches and a list of 30 other tournaments from Hopsworks. Those tournaments still need a format check before I can include their matches. Docker produced the same table from the same downloads. All 92 tests passed on my Mac and in a Linux container.

What the first comparison showed: On a separate study sample of 1,689 matches from 17 tournaments in September 2025, I compared three ways to predict a winner. Guessing 50% scored 0.25000, using both Leaders' overall records scored 0.24582, and adding their games against each other scored 0.24510. These are Brier losses: lower means the probabilities were closer to the results. I used this sample to choose my settings and kept July to September 2026 separate for a later check.

Where to start: docs/FEATURE_WALKTHROUGH.md follows one match through the pipeline. src/optcg/features/compute.py contains the calculation. The README gives the run instructions. In GitHub Actions, "Checks" runs the tests and "Daily features" runs the scheduled collection. The Actions history shows which jobs actually ran.

2 Response to MS1 reviews

1. **E1 / A1 · Check how much the direct matchup history adds**
Status: **Partly**. The direct-match rule beat the overall Leader rule in 10 of 17 tournaments. Its loss was 0.00072 lower overall, short of my practical margin of 0.0025. Across events, the loss difference ranged from −0.00685 to +0.00411. At training time, I plan to test deck composition if both decklists were available before play. For example, I could count how many cards in each list have the “Counter” component. That would be a separate input and I haven’t tested its value so far. See MS2_BASELINE_PLAN.md.

2. **A2 / B2 · Define a rule for Leaders that rarely meet**
Status: **Changed**. When two Leaders have played each other fewer than five times, I average A’s overall win rate and B’s overall loss rate. Once they’ve played each other five times, I also use the results of those games. A weight of 64 keeps a few games from dominating the estimate. Of the six weights I tested, 64 gave the lowest loss on the development games. Using the pair’s results after just one previous game scored slightly better overall than waiting for five. I’ll check both choices on later games. See MS2_BASELINE_PLAN.md and src/optcg/features/compute.py.

3. **B1 · Check both orders of a matchup and the players’ seeding**
Status: **Partly**. If the lookup gives A 60%, reversing the names gives B 40%. The same Leader card on both sides gets 50%. I’ll repeat the check once I have a trained model. I also couldn’t verify from the archive which player ranked higher before each round. See tests/test_features.py.

4. **A4 · Clarify the API terms and plan for restricted access**
Status: **Partly**. In the Limitless tournament platform’s support channel, I asked about one daily check. The reply confirmed the request rate was acceptable. If limits tighten, I can request data less often. If access stops, I would only be able to reuse downloads I’m allowed to keep. Storage and credit are still open questions. See docs/API_ACCESS_DRAFT.md.

5. **E2 and part of E1 · Explain the tool choices and what players will open**
Status: **Changed**. In my planned design, players open a web page, choose two Leaders and see the estimated win chance, how many matches it’s based on and when the data was last updated. FastAPI will supply the numbers behind that page. Hopsworks stores one feature table for training and prediction. I plan to use W&B to compare training runs and track model versions because I already know it from the NLP module. The page and training come later. See docs/FEATURE_WALKTHROUGH.md.

6. **A3 · List the dependencies and commit the locked versions**
Status: **Partly**. pyproject.toml lists the packages and uv.lock pins their versions. Docker reproduced the same table from the saved inputs. The remaining step for this review point is publishing the lockfile in the repository.